

TSKS01 Digital Communication

Lecture 11

Practical Aspects (Eye Patterns, Synchronization), Software Defined Radio

Emil Björnson

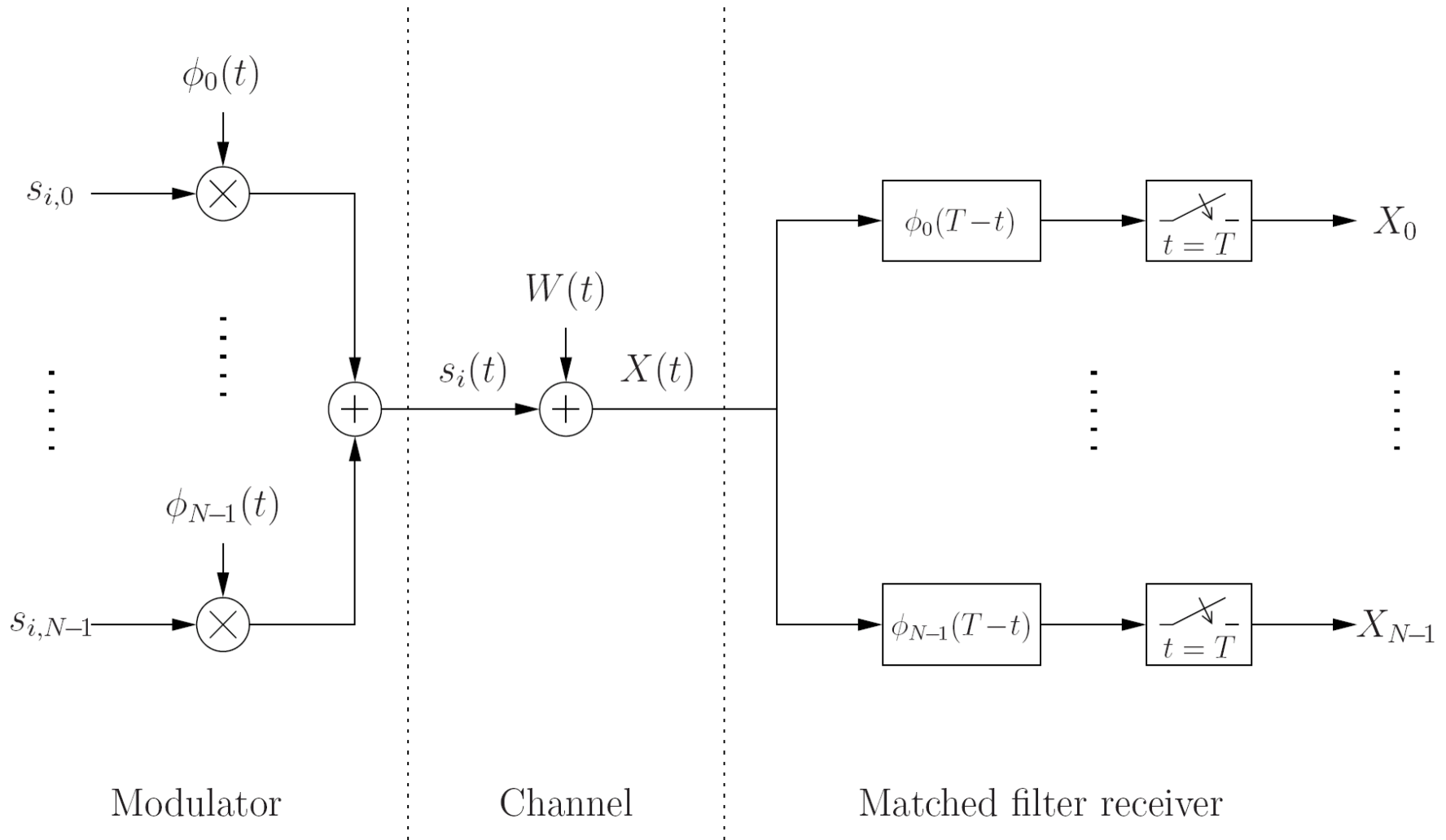
Department of Electrical Engineering (ISY)

Division of Communication Systems

Outline of the Lecture

- Practical aspects
 - Eye patterns
 - Synchronization
- Introduction to software defined radio
 - Lab system overview
 - The hardware – USRP (Universal Software Radio Peripheral)
 - The software – GNU radio

Recall: Matched Filter Receiver



Eye Patterns

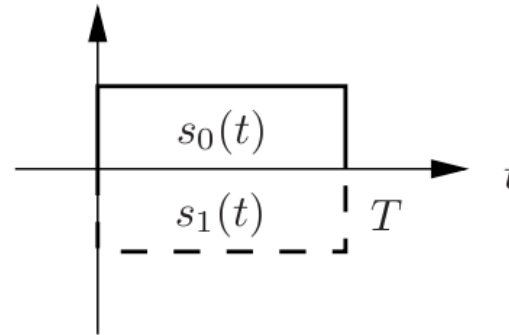
Gives a visual clue about communication quality

- Calculated:
 - All possible signals at a given place
- Measured:
 - Synchronize a memory oscilloscope
 - Generate random information
 - Measure using the oscilloscope

Calculating Eye Patterns 1(3) – Signals

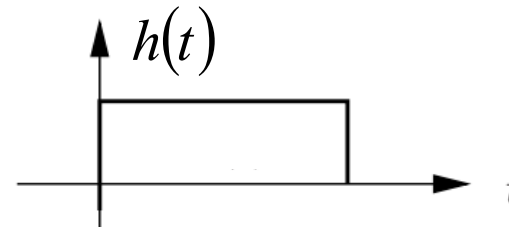
Signals:

$$s_1(t) = -s_0(t)$$



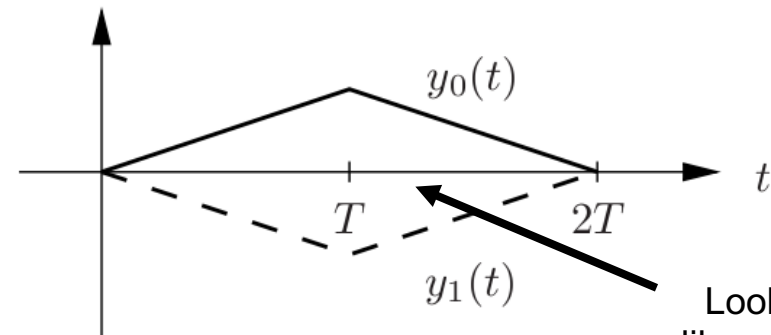
Matched filter:

$$h(t) \propto s_0(T-t)$$



Outputs:

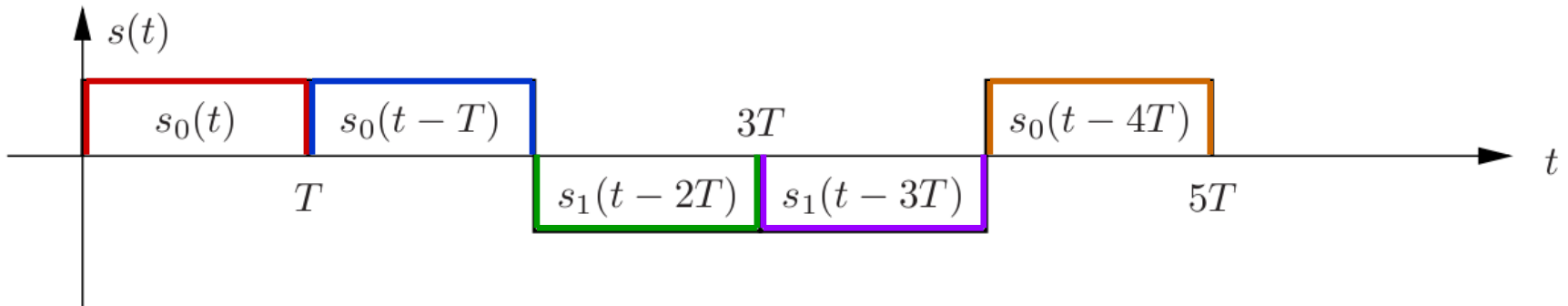
$$y_i(t) = (s_i * h)(t)$$



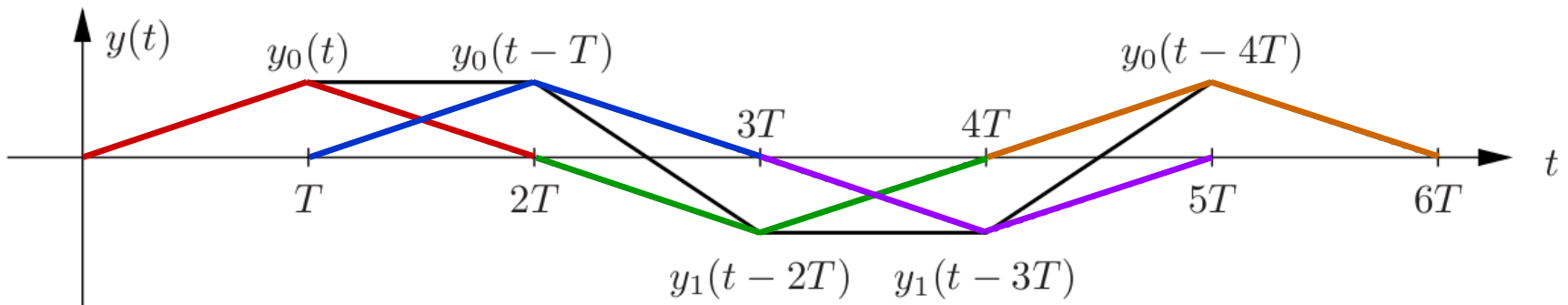
Looks slightly
like an open eye

Calculating Eye Patterns 2(3) – Sequence

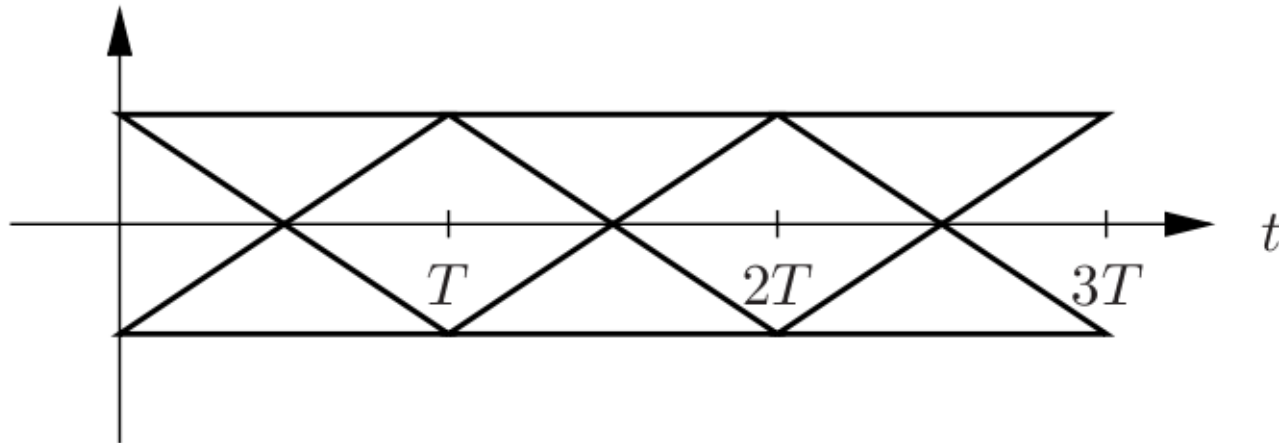
Input sequence:



Corresponding output:

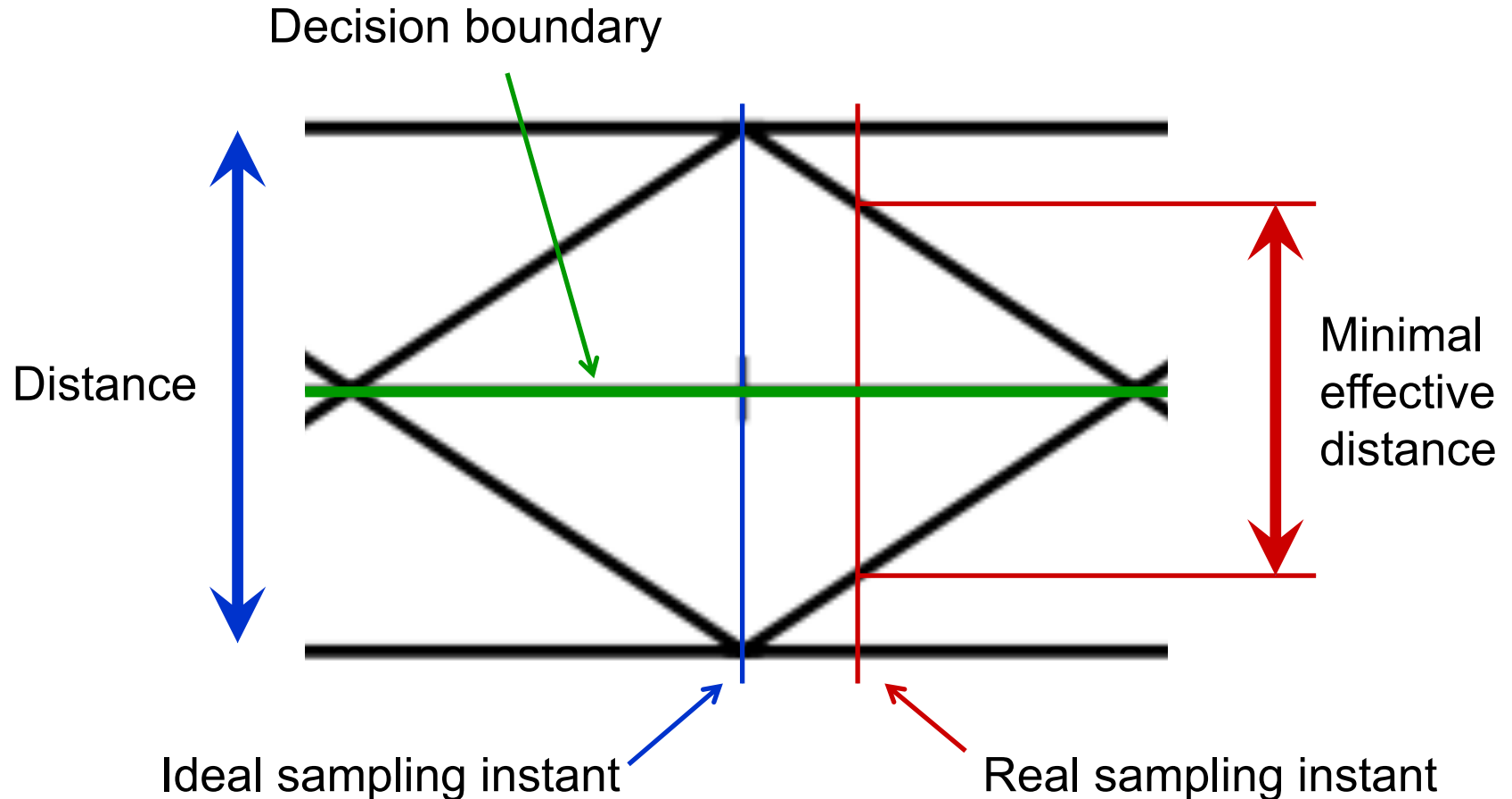


Calculating Eye Patterns 3(3) – Result



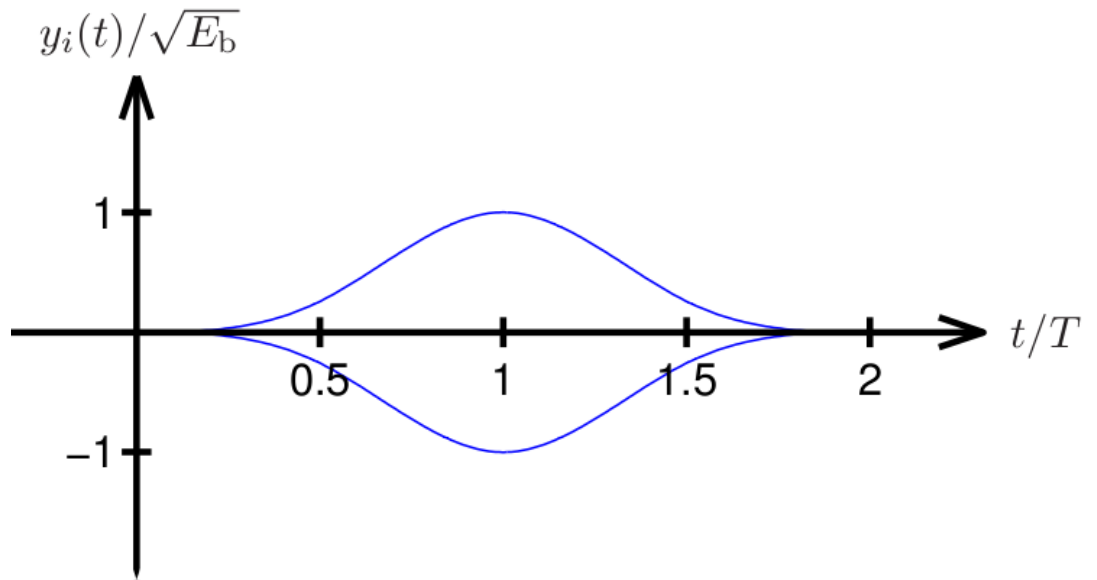
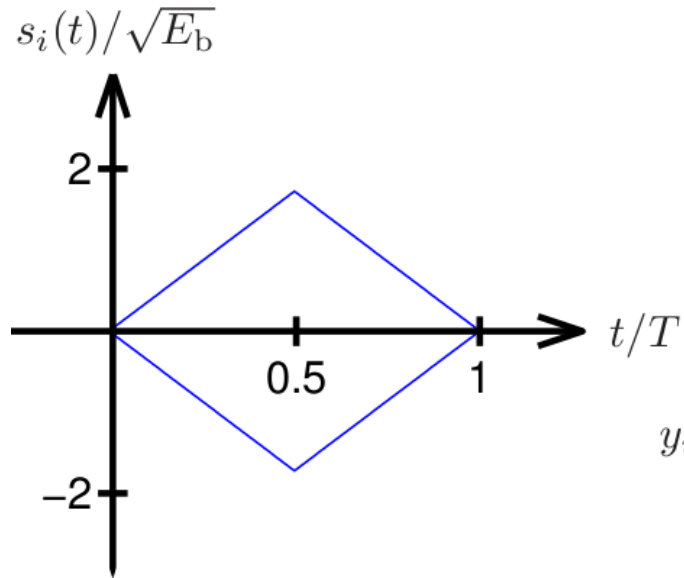
All possible output transitions during an interval of duration T , repeated over three such intervals.

Impact of Time Synchronization Errors



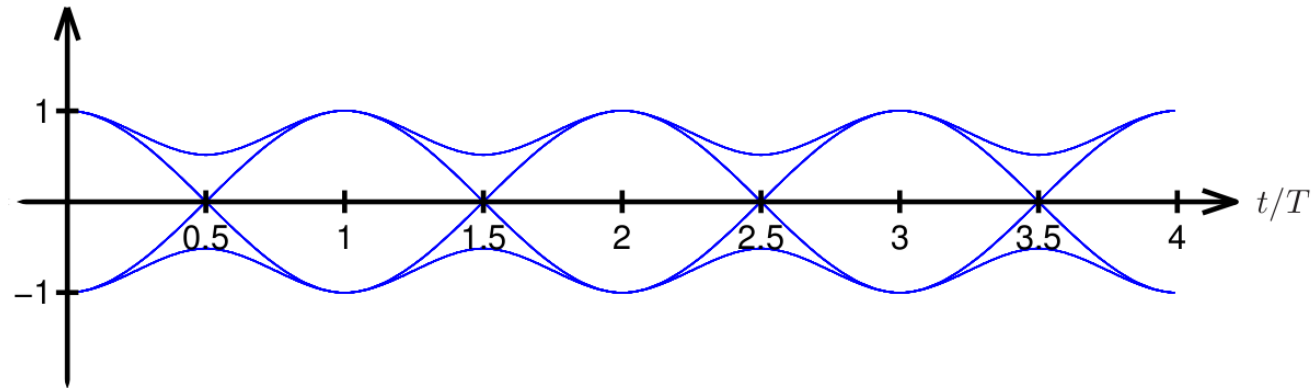
Effective distance depends on next symbol:
Not optimal to treat symbols separately

Less Sensitive Signals

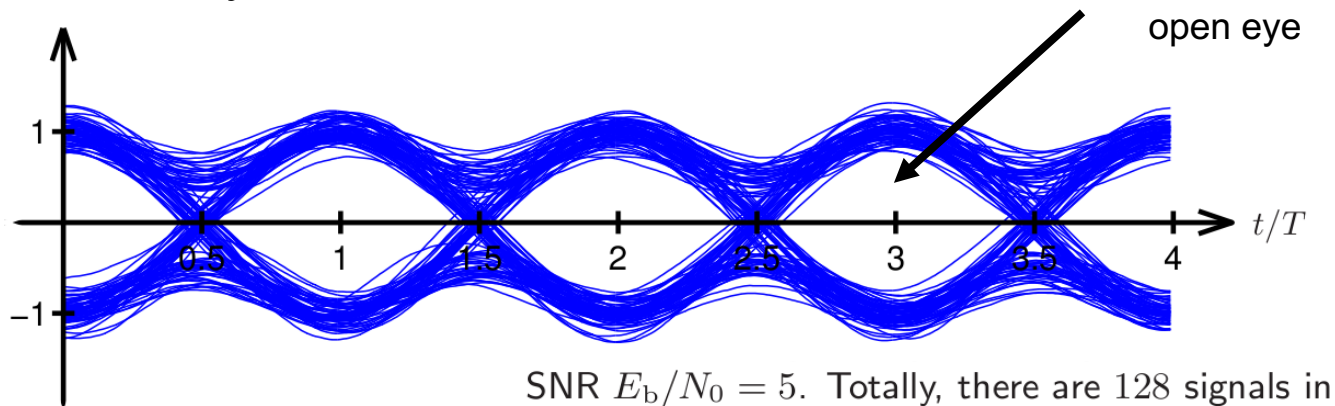


Less Sensitive Signals – Eye Pattern

Calculated Eye Pattern:

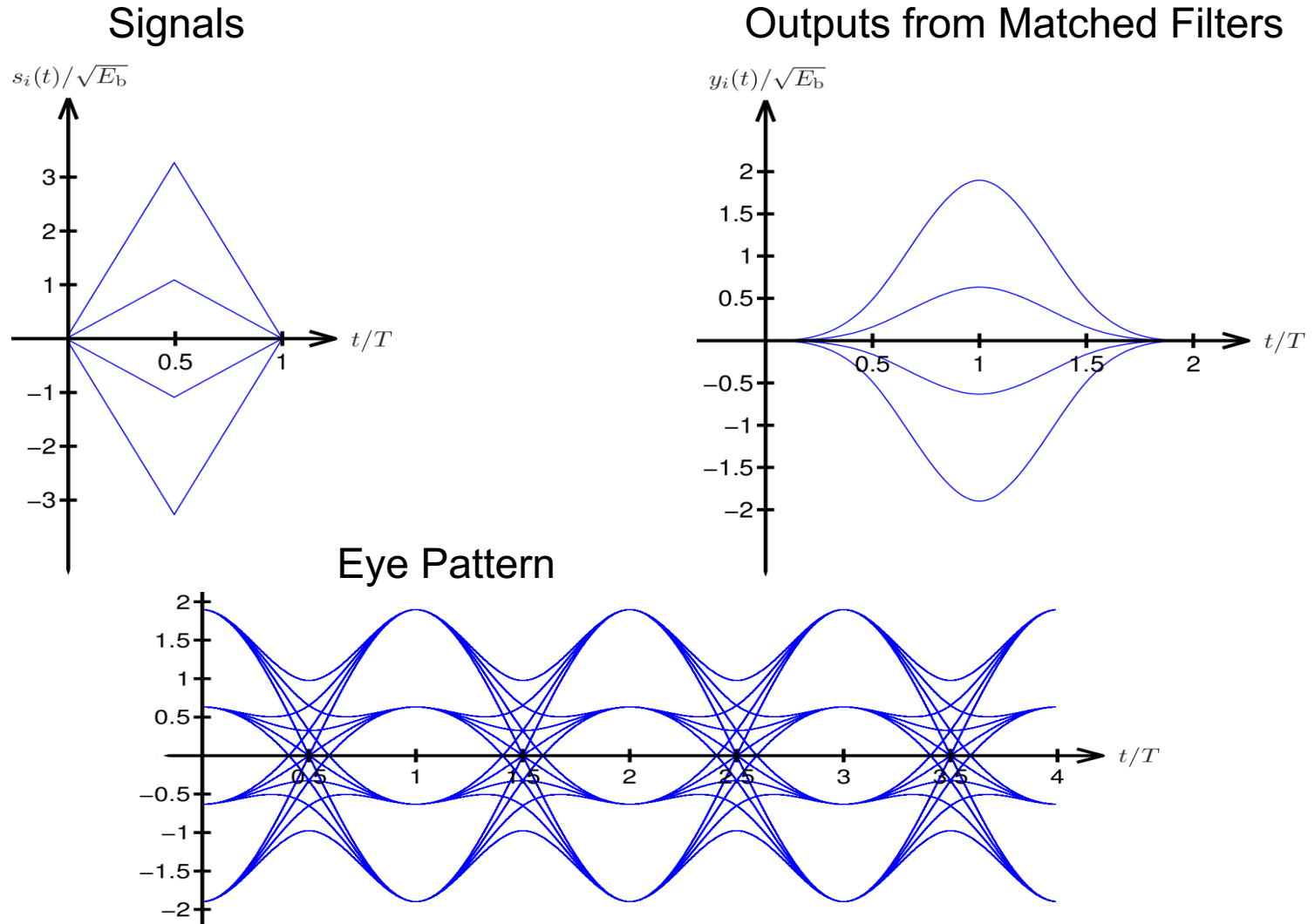


Measured Eye Pattern:

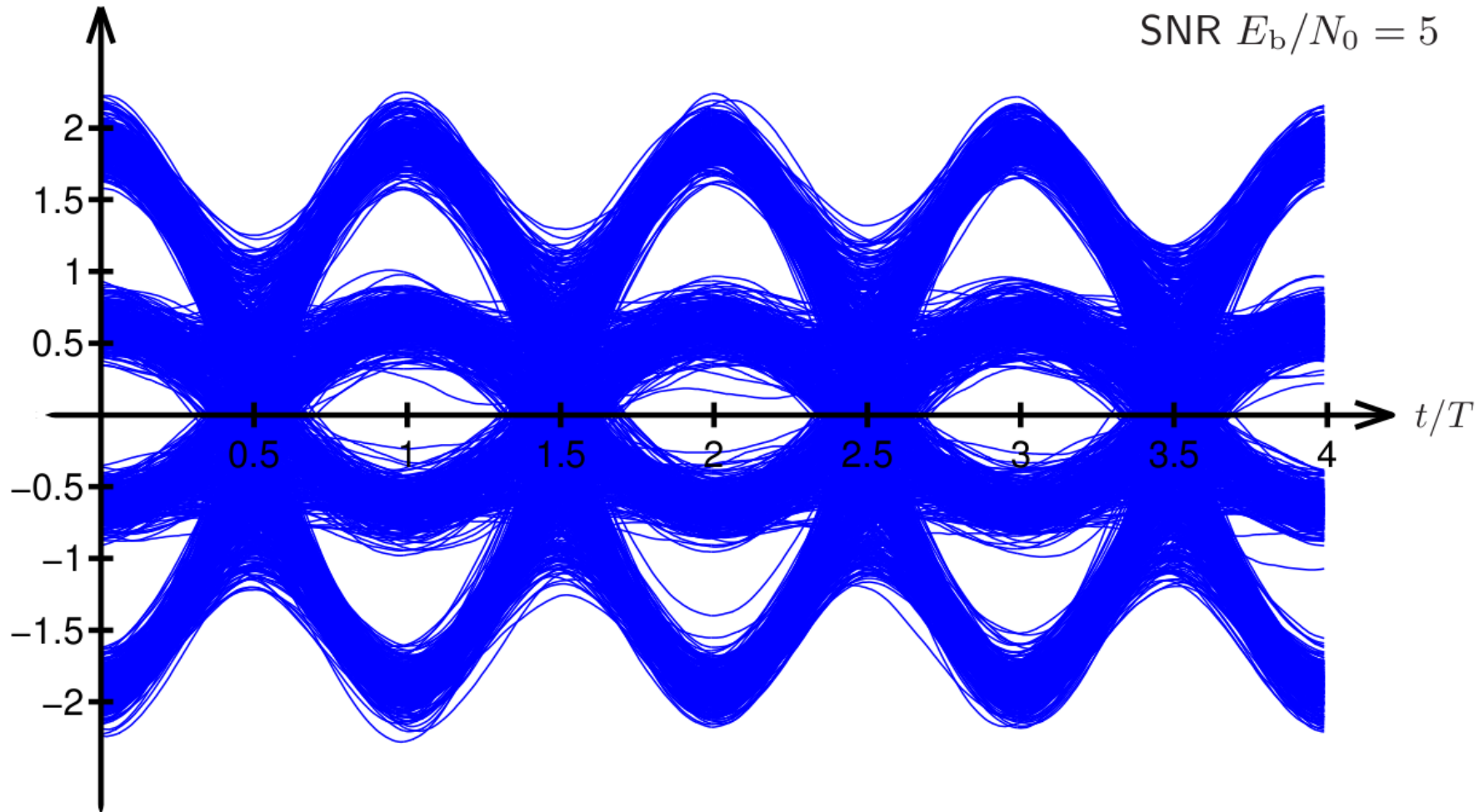


SNR $E_b/N_0 = 5$. Totally, there are 128 signals in this graph.

4-ASK – Calculated Eye Pattern



4-ASK – Measured Eye Pattern

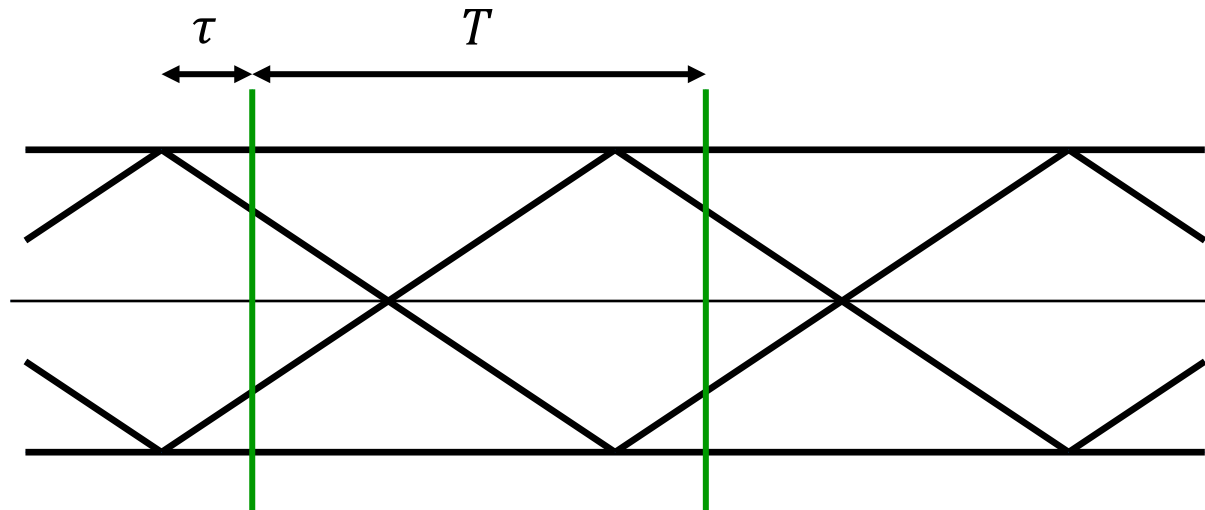


Totally, there are 1024 signals in this graph.

Timing Recovery

- How to find the right sampling time?
 - Assume: Sampling period T is known
 - Determine sampling offset τ

Samples: $y(kT + \tau)$



Timing Recovery – Mueller & Müller

Case $\tau > 0$: Samples too late

y_k : Sample

a_k : Detected (± 1)

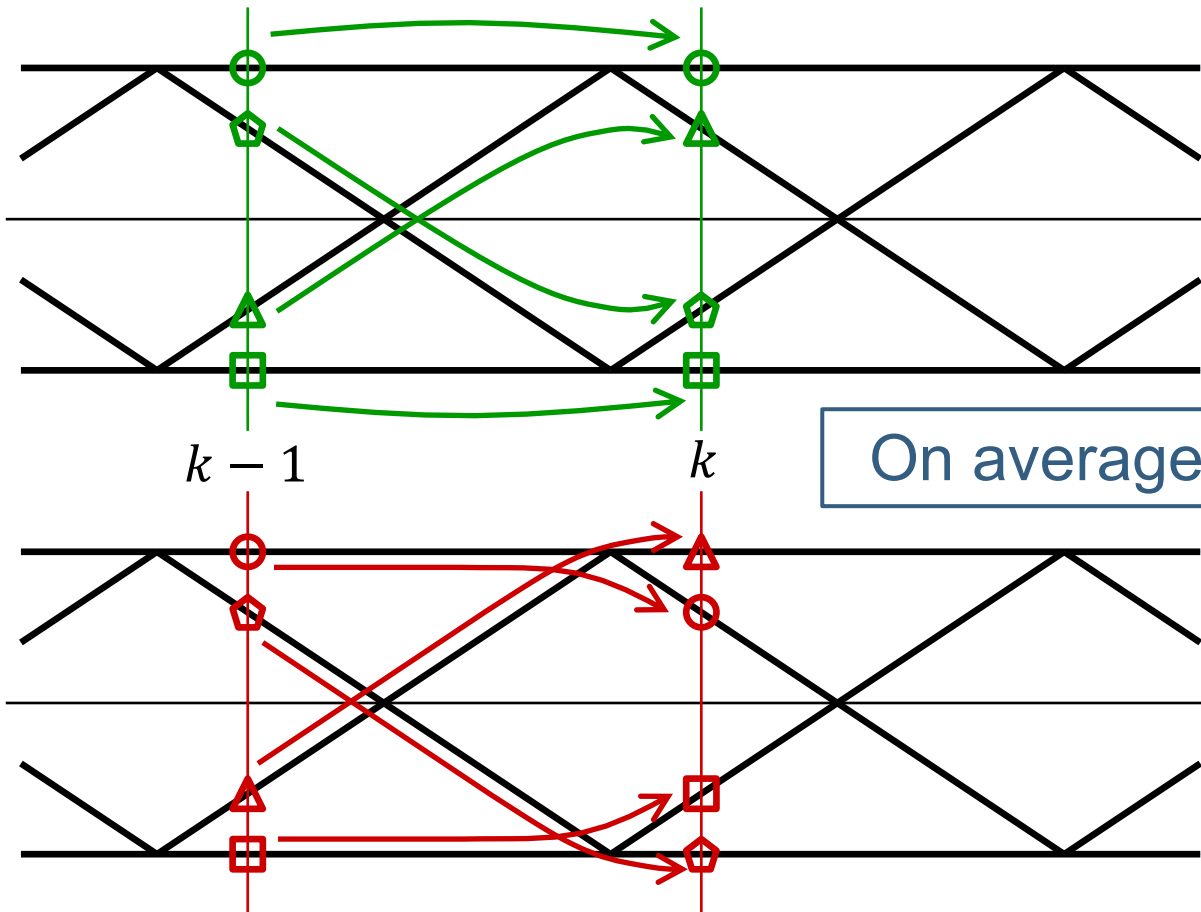
$$\Delta_k = y_k a_{k-1} - y_{k-1} a_k$$

$$\Delta_k = 0$$

On average: $\Delta_k < 0$

Tells us:
 $\tau > 0$

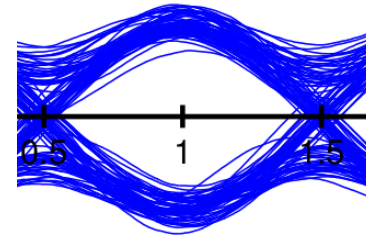
$$\Delta_k < 0$$



Timing Recovery – Summary

- Mueller & Müller algorithm
 - Compare several samples
 - Determine if samples are late ($\Delta_k < 0$) or early ($\Delta_k > 0$)
 - Adjust sampling clock to compensate ($|\Delta_k|$ indicates size of error)

Principle: Sample when the “eye” is widely open



- Dynamic algorithm
 - Run continuously to compensate for small clock variations and noise

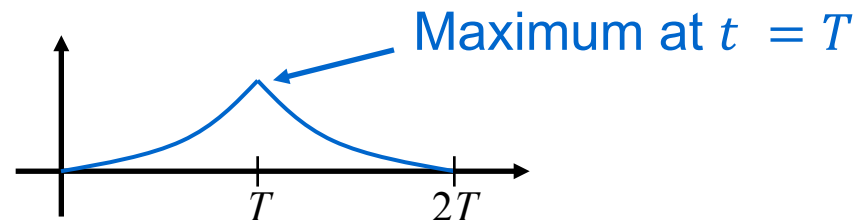
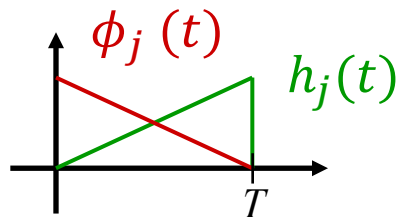
Note: T is often very small ($< 1 \mu\text{s}$!). A very small τ can be “large” enough

Carrier Synchronization: Frequency & Phase

- Recall: Matched filter receiver
 - A filter with impulse response $h_j(t) = \phi_j(T - t)$ is matched to $\phi_j(t)$

Example:

Usage $(\phi_j * h_j)(t)$:

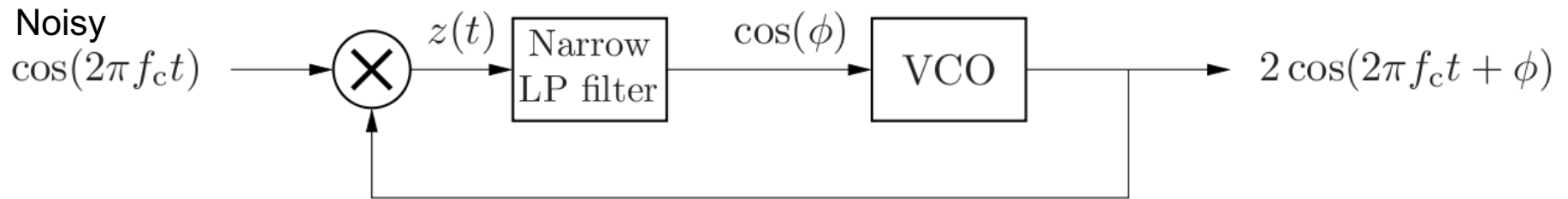


Underlying assumption: Basis functions are known!

If $\phi_j(t) = \sqrt{2/T} \cos(2\pi f_c t + \theta)$ for $0 \leq t < T$:

- Need to know f_c and θ
- Ideal values are known, but we need “exactly” same as transmitter

PLL – Phase-Locked Loop



Voltage-controlled oscillator (VCO) for f_c

- Outputs sinusoid signal with frequency and phase to get input zero
- Momentary frequency is higher when input is positive

Before LP filter:

$$z(t) = 2 \cos(2\pi f_c t) \cos(2\pi f_c t + \phi) = \cos(4\pi f_c t + \phi) + \cos(\phi)$$

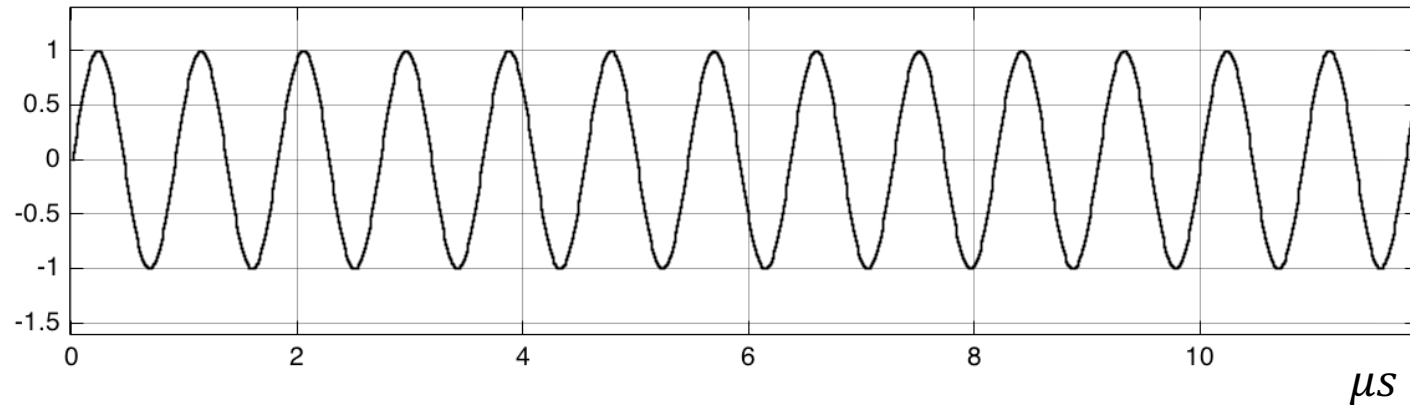
- After LP filter: $\cos(\phi)$
- PLL is stable when $\cos(\phi) \approx 0$ ($\phi \approx \pi/2$)

Always fluctuates due to noise and distortion

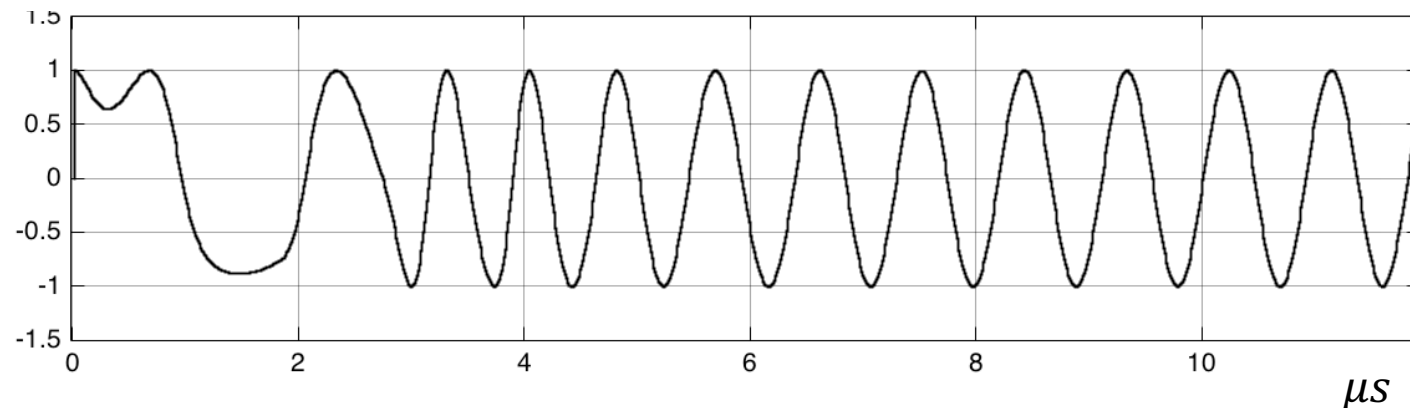
PLL - Example

Input: Sinus with $f_c = 1.1 \text{ MHz}$

(period: $0.9 \mu\text{s}$)



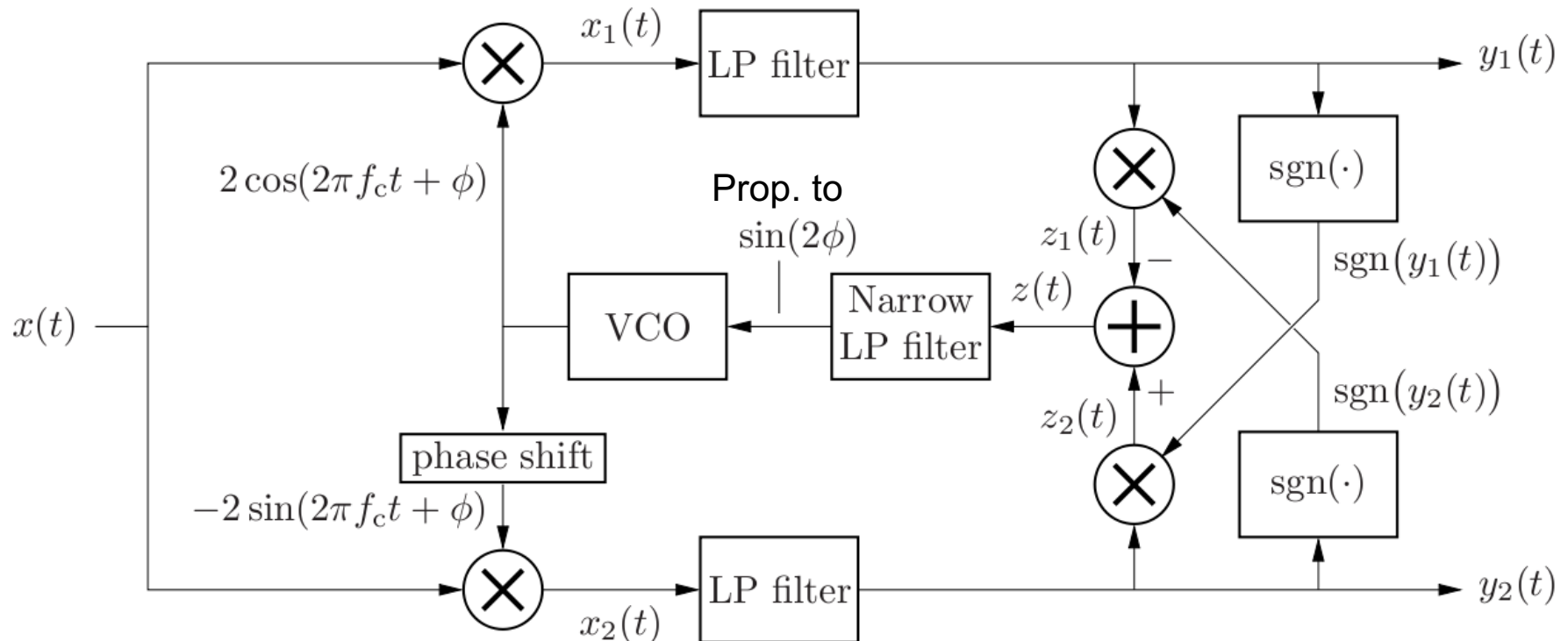
Signal from PLL with phase-shifting: Well matched after $5 \mu\text{s}$



Double Costas Loop for Detection of QPSK

Noise-free signal: $x(t) = x_I(t) \cos(2\pi f_c t) - x_Q(t) \sin(2\pi f_c t)$

Takes values ± 1



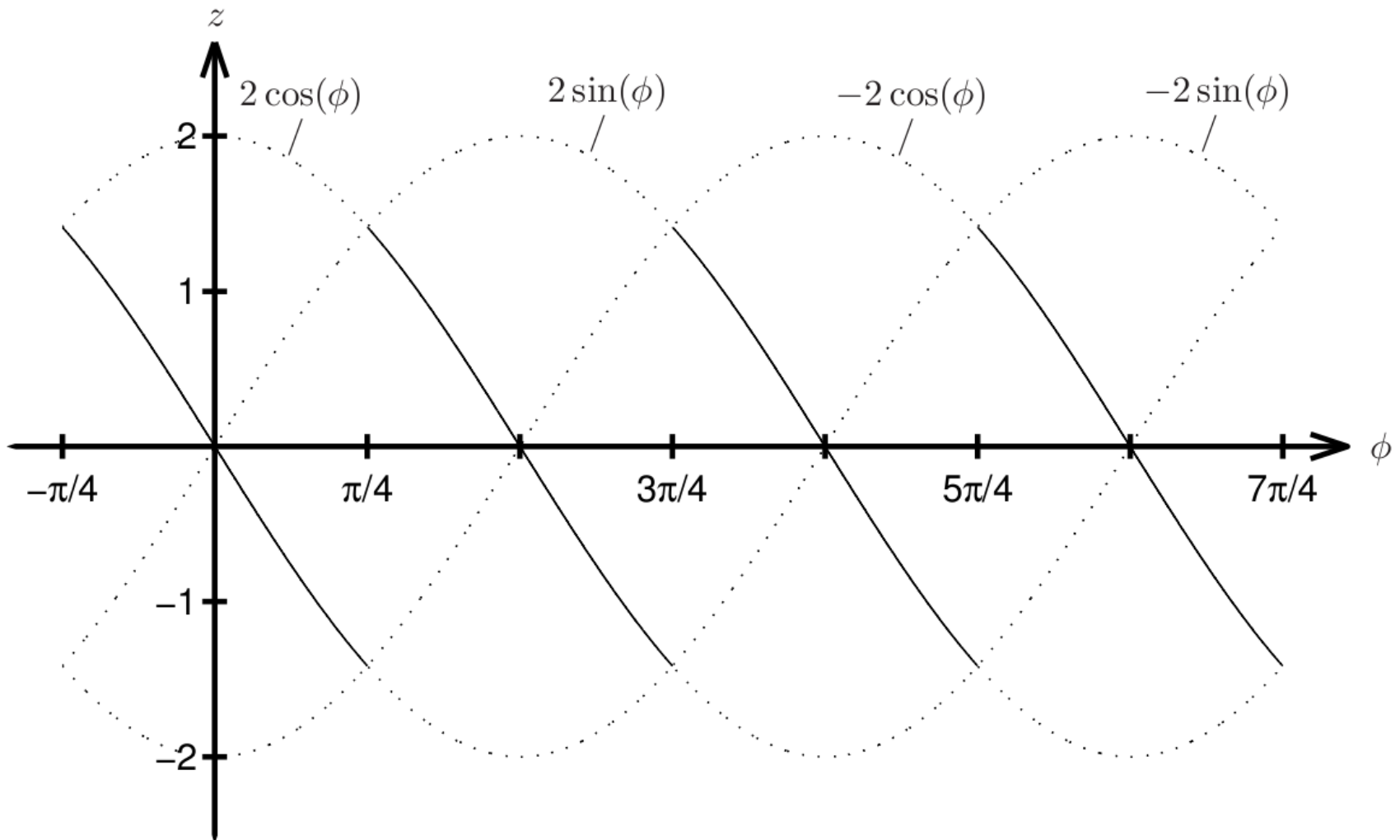
Analysis of Double Costas Loop 1(3)

Interval $\phi \bmod 2\pi$	x_I, x_Q	$\text{sgn}(y_1),$ $\text{sgn}(y_2)$	z_1	z_2	z
$(-\frac{\pi}{4}, \frac{\pi}{4})$		x_I, x_Q			
	+, +	+, +	$\cos(\phi) + \sin(\phi)$	$-\sin(\phi) + \cos(\phi)$	$-2 \sin(\phi)$
	+, -	+, -	$-\cos(\phi) + \sin(\phi)$	$-\sin(\phi) - \cos(\phi)$	$-2 \sin(\phi)$
	-, +	-, +	$-\cos(\phi) + \sin(\phi)$	$-\sin(\phi) - \cos(\phi)$	$-2 \sin(\phi)$
	-, -	-, -	$\cos(\phi) + \sin(\phi)$	$-\sin(\phi) + \cos(\phi)$	$-2 \sin(\phi)$
$(\frac{\pi}{4}, \frac{3\pi}{4})$		$x_Q, -x_I$			
	+, +	+, -	$-\cos(\phi) - \sin(\phi)$	$-\sin(\phi) + \cos(\phi)$	$2 \cos(\phi)$
	+, -	-, -	$-\cos(\phi) + \sin(\phi)$	$\sin(\phi) + \cos(\phi)$	$2 \cos(\phi)$
	-, +	+, +	$-\cos(\phi) + \sin(\phi)$	$\sin(\phi) + \cos(\phi)$	$2 \cos(\phi)$
	-, -	-, +	$-\cos(\phi) - \sin(\phi)$	$-\sin(\phi) + \cos(\phi)$	$2 \cos(\phi)$

Analysis of Double Costas Loop 2(3)

Interval $\phi \bmod 2\pi$	x_I, x_Q	$\text{sgn}(y_1),$ $\text{sgn}(y_2)$	z_1	z_2	z
$(\frac{3\pi}{4}, \frac{5\pi}{4})$		$-x_I, -x_Q$			
	$+, +$	$-, -$	$-\cos(\phi) - \sin(\phi)$	$\sin(\phi) - \cos(\phi)$	$2 \sin(\phi)$
	$+, -$	$-, +$	$\cos(\phi) - \sin(\phi)$	$\sin(\phi) + \cos(\phi)$	$2 \sin(\phi)$
	$-, +$	$+, -$	$\cos(\phi) - \sin(\phi)$	$\sin(\phi) + \cos(\phi)$	$2 \sin(\phi)$
	$-, -$	$+, +$	$-\cos(\phi) - \sin(\phi)$	$\sin(\phi) - \cos(\phi)$	$2 \sin(\phi)$
$(\frac{5\pi}{4}, \frac{7\pi}{4})$		$-x_Q, x_I$			
	$+, +$	$-, +$	$\cos(\phi) + \sin(\phi)$	$\sin(\phi) - \cos(\phi)$	$-2 \cos(\phi)$
	$+, -$	$+, +$	$\cos(\phi) - \sin(\phi)$	$-\sin(\phi) - \cos(\phi)$	$-2 \cos(\phi)$
	$-, +$	$-, -$	$\cos(\phi) - \sin(\phi)$	$-\sin(\phi) - \cos(\phi)$	$-2 \cos(\phi)$
	$-, -$	$+, -$	$\cos(\phi) + \sin(\phi)$	$\sin(\phi) - \cos(\phi)$	$-2 \cos(\phi)$

Analysis of Double Costas Loop 3(3)



Introduction: Software Defined Radio (SDR)

- Idea: As much as possible in software – Provides flexibility
- Wide-band sampling – Large bandwidths.
- Vision: Flexible radio equipment, easy updates.
- The industry is interested in SDR.

Lab System Overview: USRP

– Universal Software Radio Peripheral

- Made by Ettus Research
- Mostly open-source design
- Suitable for prototyping
- Used for SDR research
- Normal PC used for signal processing
- USB interface to PC
- Programmable FPGA for base-band manipulations
- Fixed replaceable RF frontends



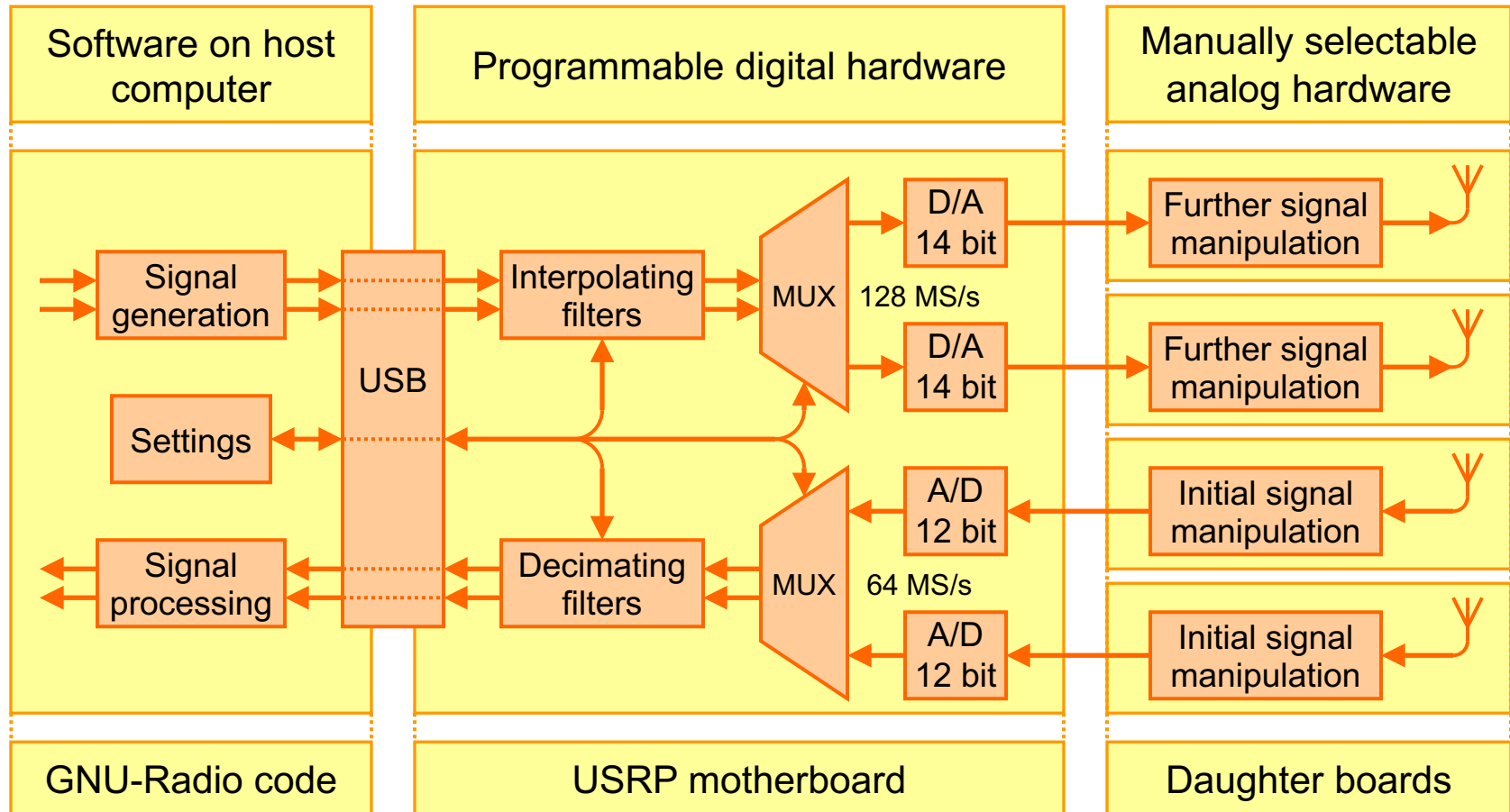
Lab System Overview: GNU Radio

- Open-source software framework for SDR
- Runs on standard Linux/Windows computers
- Flowgraphs define signal processing chains
- Primitive blocks written in C++
- Flowgraph models in Python



Overview: Interaction

Flexibility: Do as much of the signal processing as possible in software.



First Impressions

Documentation?

SWIG

Python

USB

Yikes!!

C++

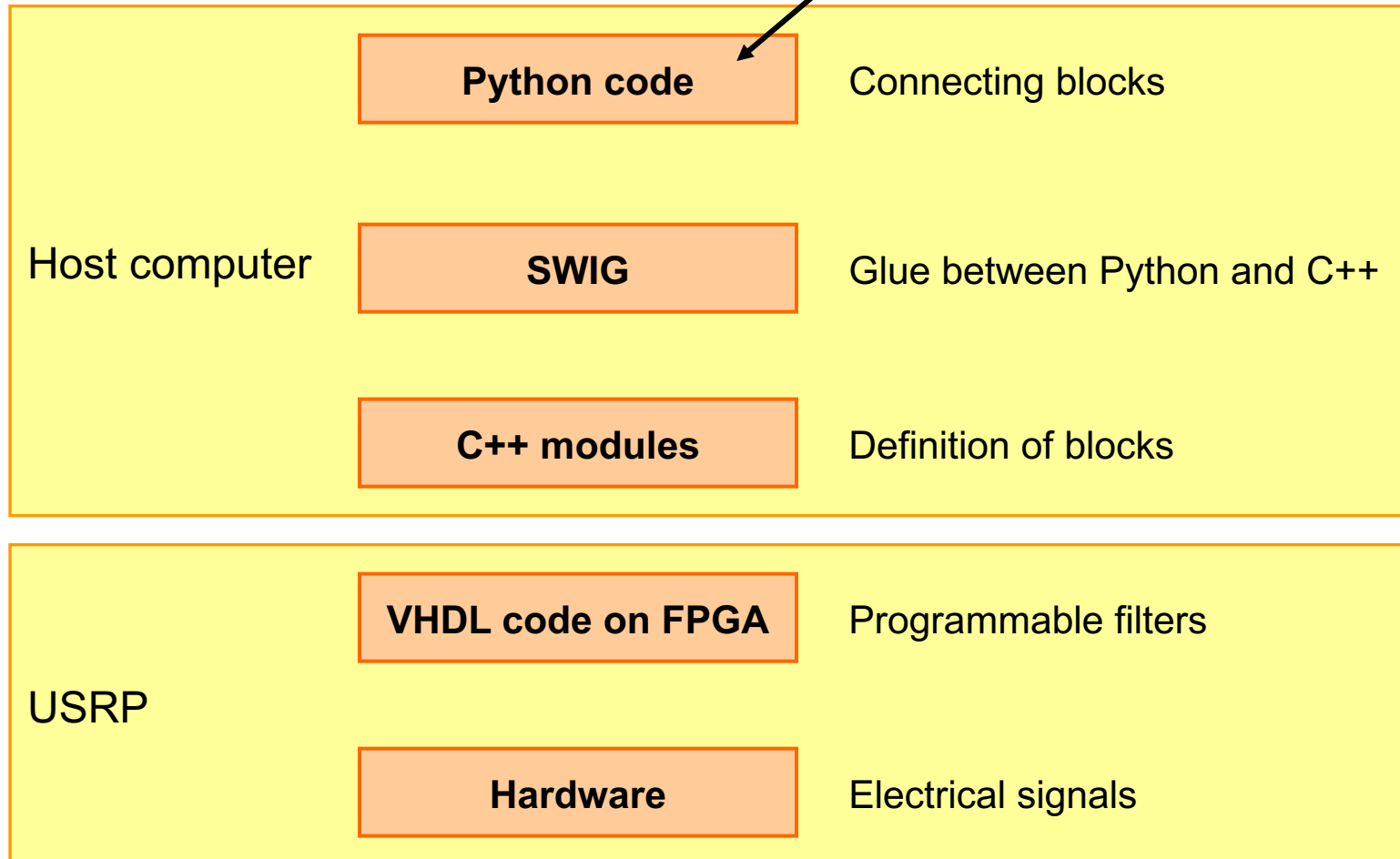
VHDL

Hardware

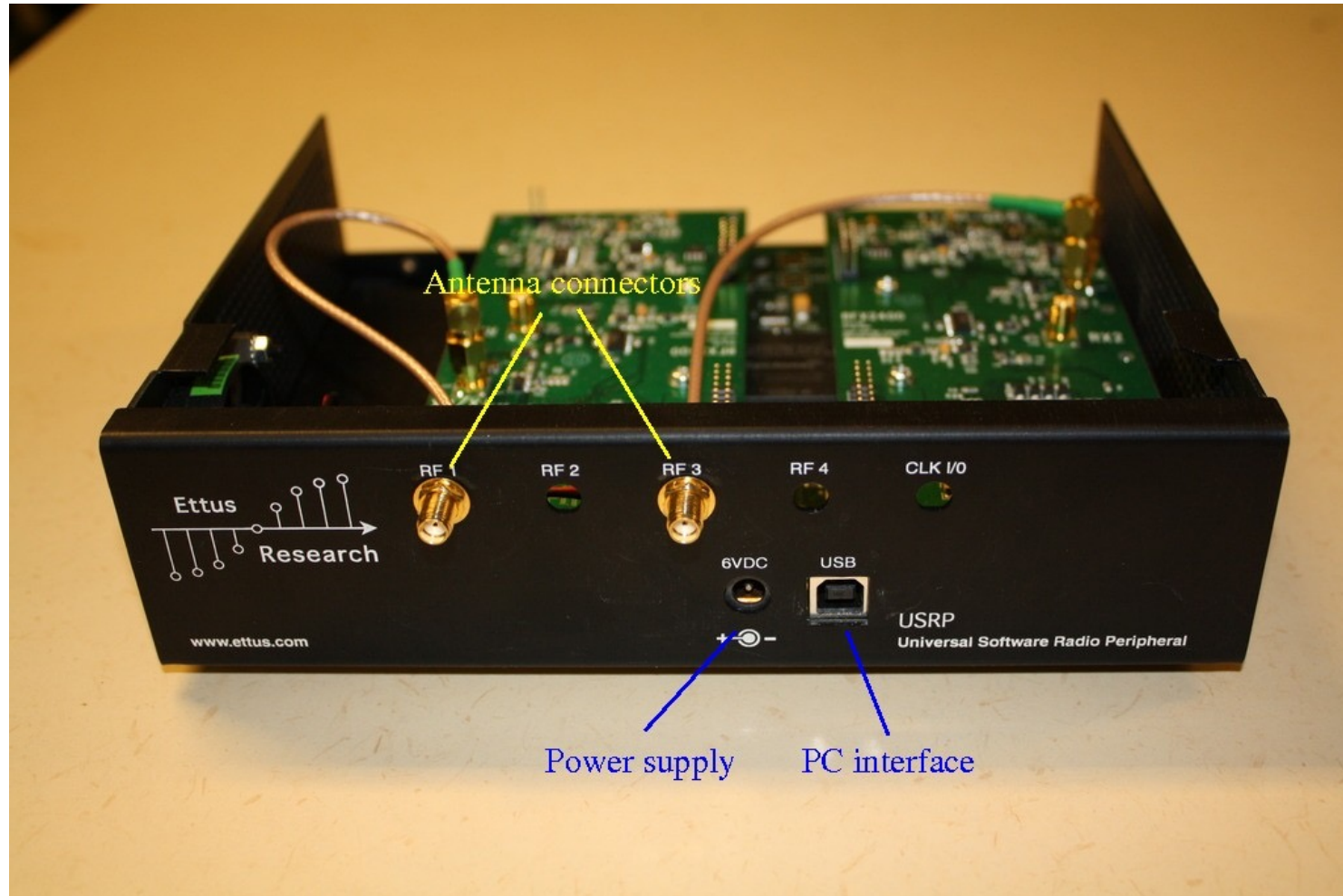
Tutorials?

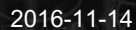
Abstraction Layers

Some of this is sufficient
at the lab exercises

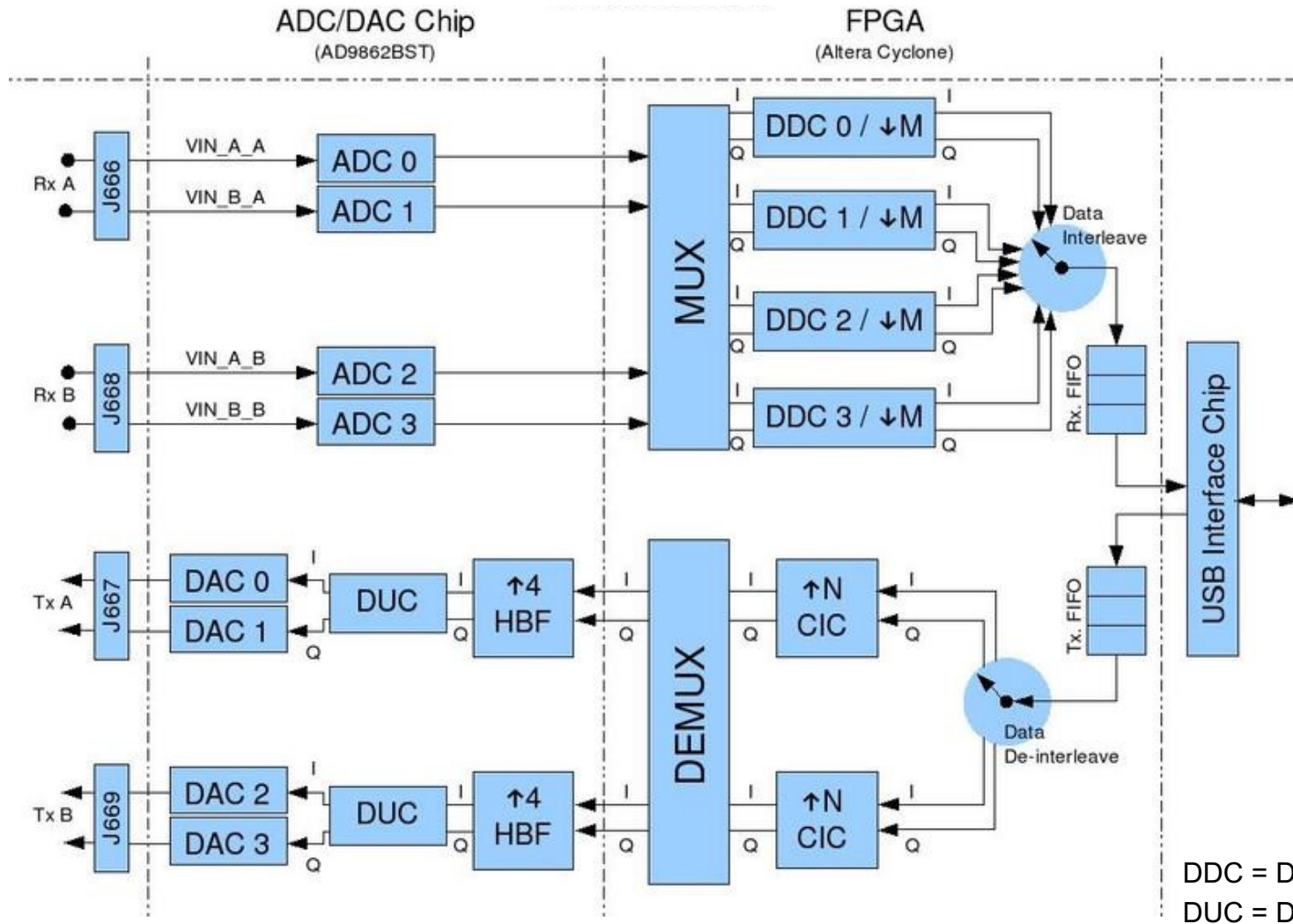


The USRP – Front Panel

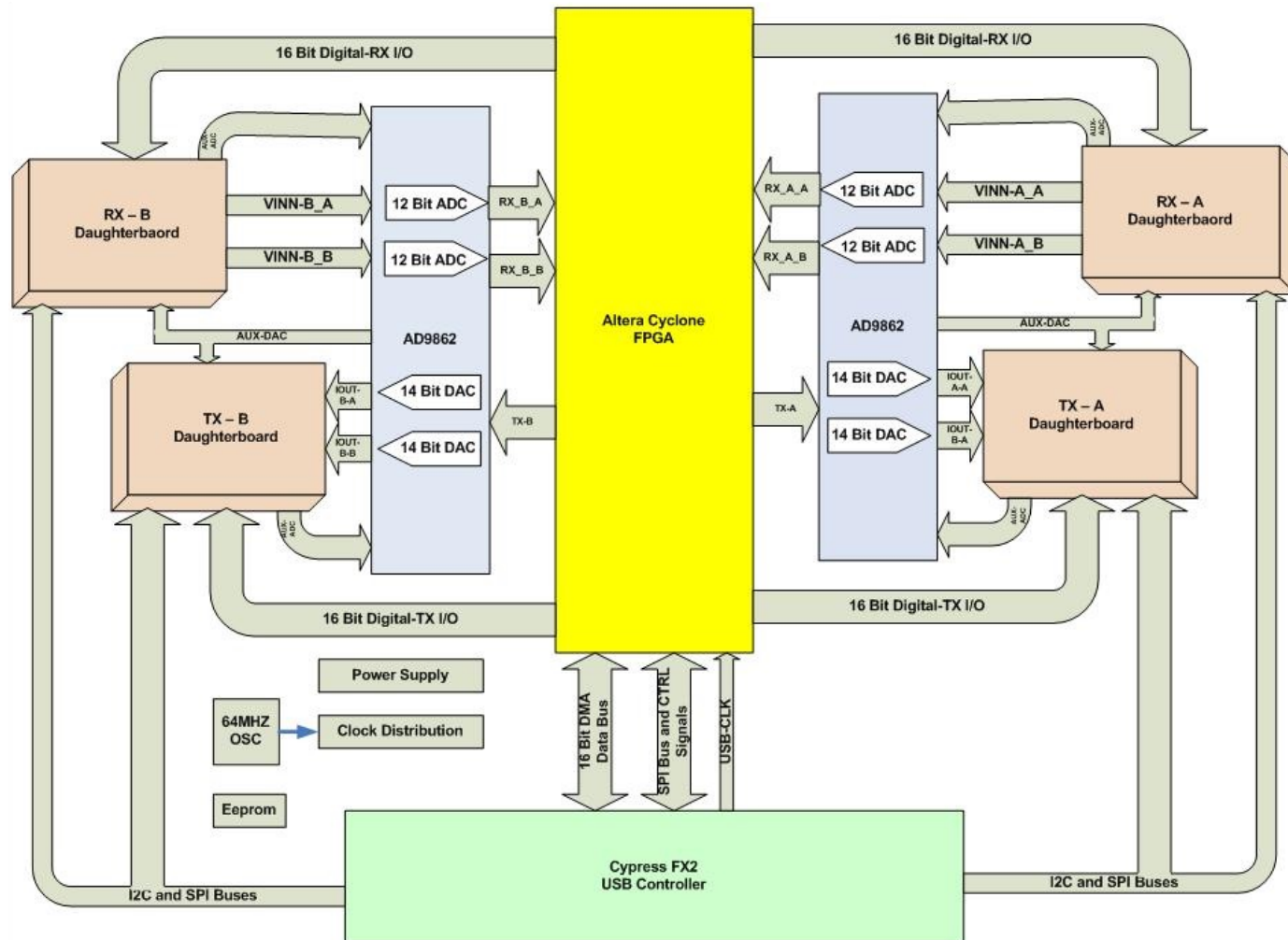




USRP – Mother Board Functionality

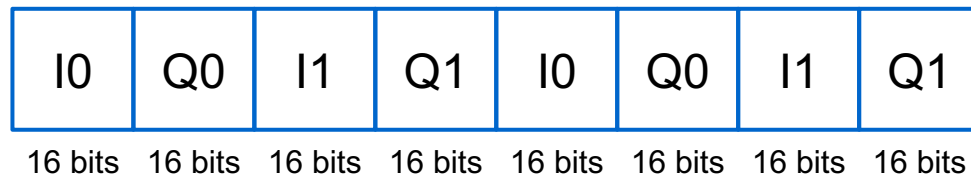


USRP – Mother Board Block Diagram



The USRP – USB Interface

- USB capabilities
 - USB 2.0: 480 Mbit/s
 - FPGA firmware: 32 MB/s (= 256 Mbit/s) useful data rate
 - 32 bit complex samples (16 bits each for real/imaginary)
 - Limit: 8 MS/s (combined for all tx/rx streams on one computer)
- USB transmission order:



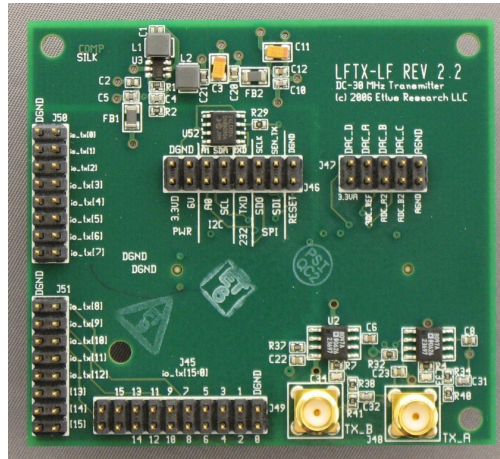
The USRP – Daughter Boards 1(2)

- Base-band boards: BasicTX/BasicRX, LFTX/LFRX
 - Bandwidth: 250 MHz (Basic), 30 MHz (LF)
 - Can not connect directly to antenna
- WBX - Wide-band transceiver
 - 50 MHz – 2.2 GHz
 - Relatively poor performance
- TVRX - 50 – 860 MHz receiver
 - Bandwidth: 6 MHz
 - Suitable for receiving TV signals
- RFX-series - General full-duplex transceivers
 - Bandwidth: 30 MHz
 - Bands: 400–500 MHz, 750–1050 MHz, 1150–1450 MHz, 1.5–2.1 GHz, 2.3–2.9 GHz

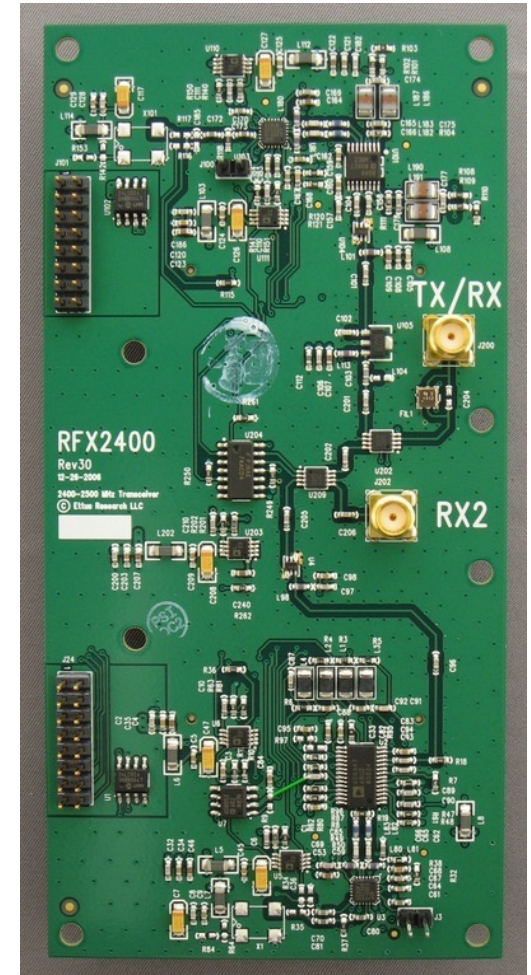
Used in lab
exercises

The USRP – Daughter Boards 2(2)

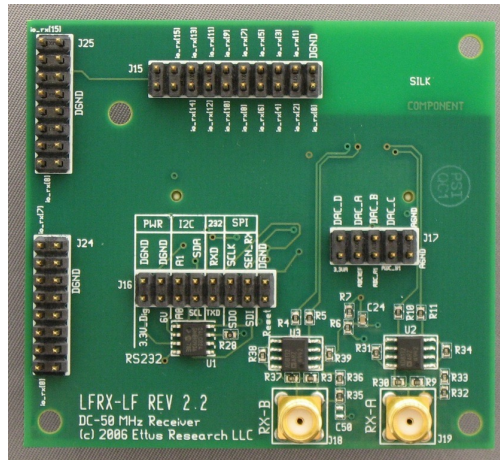
LFTX



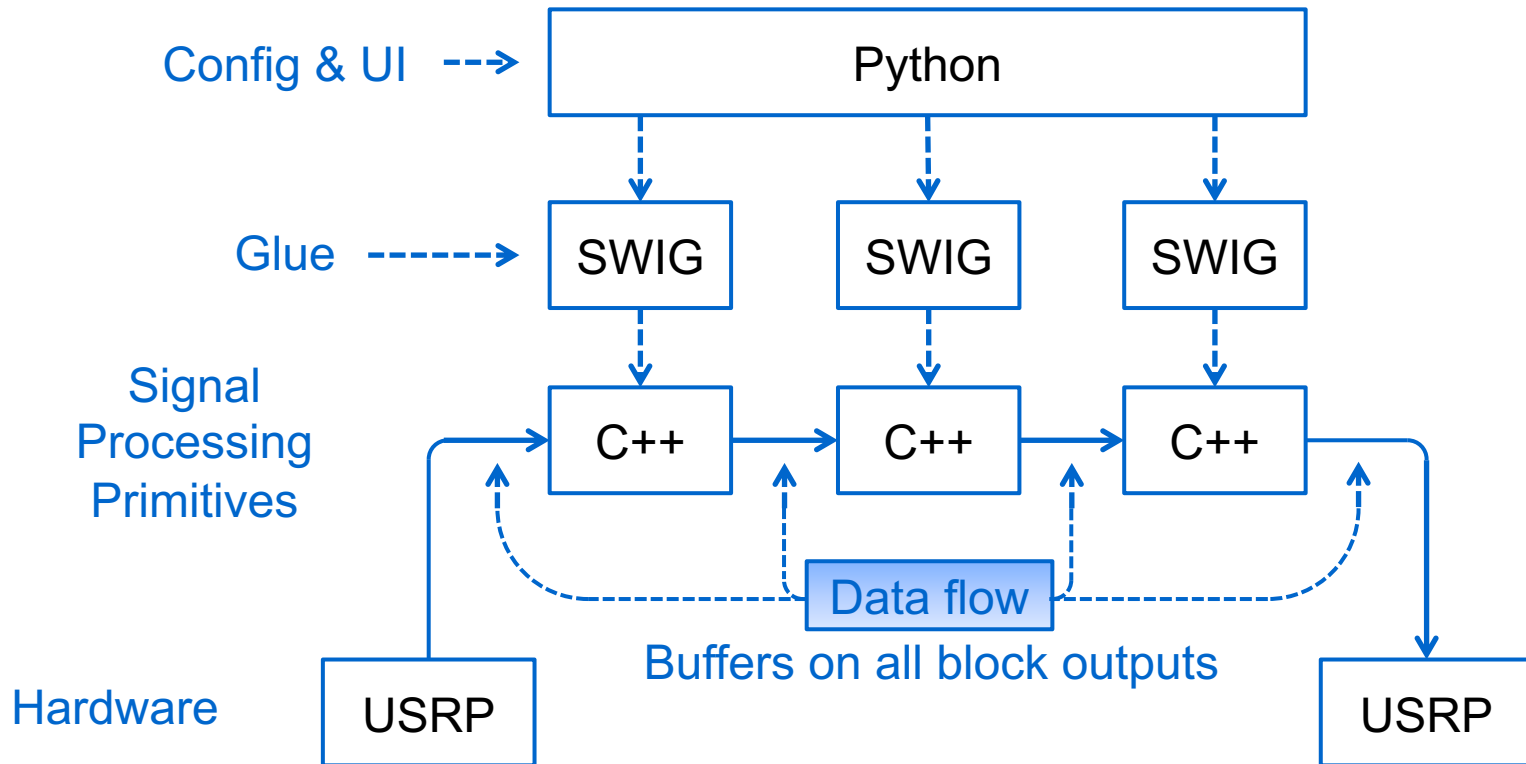
RFX-2400



LFRX

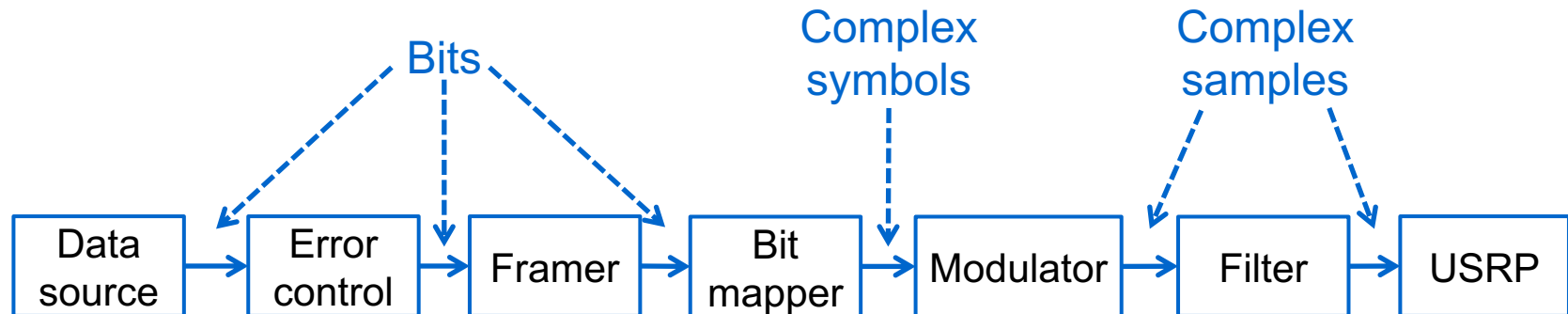


GNU Radio – Software Layers



GNU Radio – Flowgraphs

Data types may vary along a flowgraph.



Data types must match in each connection.

GNU Radio – Hello World!

```
from gnuradio import gr
from gnuradio import audio
```

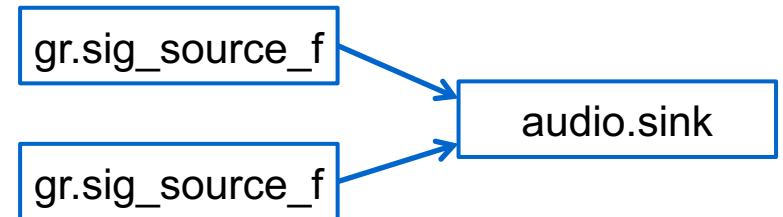
```
class my_top_block(gr.top_block):
    def __init__(self):
        gr.top_block.__init__(self)
```

```
        sample_rate = 48000
        ampl = 0.1
```

```
        src0 = gr.sig_source_f(sample_rate, gr.GR_SIN_WAVE, 350, ampl)
        src1 = gr.sig_source_f(sample_rate, gr.GR_SIN_WAVE, 440, ampl)
        dst = audio.sink(sample_rate)
```

```
        self.connect(src0, (dst, 0))
        self.connect(src1, (dst, 1))
```

```
tb = my_top_block()
tb.run()
```



GNU Radio – Block Naming Convention

- Block naming: *lib.blocktype_suffix*
- *lib* is the library:
 - gr** for most primitive signal processing blocks
 - blks2** for high-level signal processing blocks
 - usrp** for USRP interfacing blocks
 - audio** for audio sources and sinks
 - qtgui** for GUI library (relatively bare)
 - wxgui** for old GUI library
 - Projects usually use their own library
- *suffix* determines port types
 - b** for byte, **s** for 16-bit int, **i** for 32-bit int, **f** for 32-bit float, **c** for 64-bit complex
 - x** for sources and sinks
 - xx** for two-port blocks
 - xxx** for parameterized blocks
 - (e.g. **ccf** for FIR filter with complex data and float coefficients)
 - vx**, **vxx** for blocks with vector ports

GNU Radio – Some Useful Blocks

- `gr.noise_source_f` :
Generates random float samples with specified distribution.
- `gr.add_cc` :
Adds two streams of complex samples.
- `gr.nlog10_ff` :
Computes dB of a stream of floats.
- `gr.fir_filter_ccc` :
FIR filter with complex input, output and coefficients.
- `audio.source` :
Stream of floats from line-in/mic.
- `usrp.sink_c` :
USRP sink for transmitting a complex stream.
- `qtgui.sink_c` :
GUI element with oscilloscope, FFT and constellation diagram.

GNU Radio – Connecting Blocks

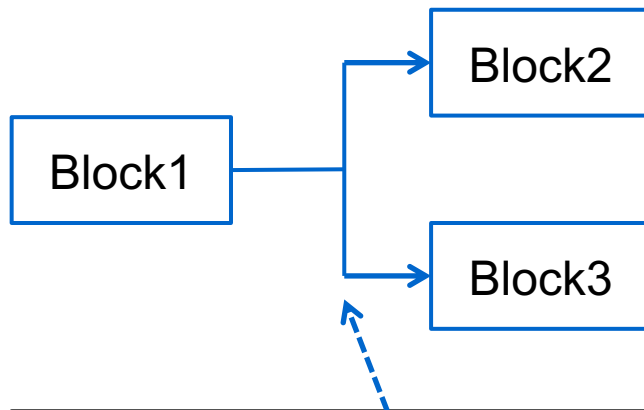


```
self.connect(Block1, Block2, Block3)
```

Equivalent:

```
self.connect(Block1, Block2)
```

```
self.connect(Block2, Block3)
```



```
self.connect(Block1, Block2)
```

```
self.connect(Block1, Block3)
```

Restriction: Both Block2 and Block3 must consume the data.
If Block2 does not, then Block3 will stall.

GNU Radio – Run-Time Behaviour

- Heavily threaded
 - Each block runs in a separate thread
 - Most blocks have thread-safe configuration functions
- Starting a flow graph:
 - `gr.top_block.run()`:
Spawns threads, waits for completion
 - `gr.top_block.start()`:
Spawns threads and returns
- Stopping it:
 - `gr.top_block.stop()`

```
from gnuradio import gr
from gnuradio import audio
class my_top_block(gr.top_block):
    def __init__(self):
        gr.top_block.__init__(self)
        self.src = gr.sig_source_f(48000,
                                    gr.GR_SIN_WAVE, 350, 0.1)
        self.dst = audio.sink(48000)
        self.connect(self.src, self.dst)

tb = my_top_block()
tb.start()
f = 440
while f>0:
    c = raw_input("Enter frequency: ")
    f = float(c)
    tb.src.set_frequency(f)
tb.stop()
```

Python

- Python is
 - A full-blown high-level programming language
 - Half-compiled, half-interpreted
- Python supports
 - Object-oriented programming
 - Procedural programming
 - Functional programming (to some extent)
 - Interactive testing of code
- Python is used for
 - Scripting
 - Web-development
 - Full-blown computer applications with GUIs.
- Python is used by
 - Google, YouTube, NASA, Disney, Hubble, CommSys, ...

Intro: Title page

```
# Title page:
basisStrings=(
    '', '', '',
    'Introduction to Python', '',
    '', 'Mikael Olofsson',
    'mikael@isy.liu.se', '',
    'Communication Systems',
    'ISY', '',
    '2011-09-20'
)
centeredStrings=[
    x.center(38)
    for x in basisStrings
]
print '\n'.join(centeredStrings)
```

Introduction to Python

Mikael Olofsson
mikael@isy.liu.se

Communication Systems
ISY

2011-09-20



Run

Clear



Intro: Outline

```
# Outline:
basisStrings=(
    '', '', '',
    'Outline', '-----',
    '', 'Introduction',
    '', 'Python basics',
    '', 'Classes and objects',
#   '', 'Standard modules',
#   '', 'Numeric',
#   '', 'matplotlib',
#   '', 'SciPy',
)
centeredStrings=[
    x.center(38)
    for x in basisStrings
]
print '\n'.join(centeredStrings)
```

```
Outline
-----

Introduction

Python basics

Classes and objects
```

Intro: Properties

```
# Properties
basisStrings=(
    '',
    'Properties of Python',
    '-----',
    '', 'Object oriented',
    '', 'Dynamically typed',
    '', 'Pure syntax',
    '', '"Half interpreted"',
    '', ' Automatical garbage collection',
    '', 'Interactive environment', '',
    ' o-----o',
    '| Nice atmosphere |',
    '| at com.lang.python |',
    ' o-----o',
)

centeredStrings=[
    x.center(38)
    for x in basisStrings
]

print '\n'.join(centeredStrings)
```

Properties of Python

Object oriented

Dynamically typed

Pure syntax

"Half interpreted"

Automatical garbage collection

Interactive environment

```
o-----o
| Nice atmosphere |
| at com.lang.python |
o-----o
```



Run

Clear



Basics: Basic data types

```
—  
# Basic data types:  
a = 2                # integers  
b = 3.0              # floats  
c = complex(2,3)     # complex number  
d = 'abc'            # string  
e = (1,'abc',4.0)     # tuple (immutable)  
f = [1,'abc',4.0]     # list (mutable)  
g = {'c':2,'a':3}     # Dictionary  
h = None             # None  
  
print a, b, c, d  
print e, f  
print g, h  
  
print e[0], e[1], e[2]  
print e[-1], e[-2], e[-3]  
print f[1], f[-1], g['a']  
print d, repr(d)
```

```
2 3.0 (2+3j) abc  
(1, 'abc', 4.0) [1, 'abc', 4.0]  
{'a': 3, 'c': 2} None  
  
1 abc 4.0  
4.0 abc 1  
abc 4.0 3  
abc 'abc'
```

Basics: Handle lists

```
# Handle lists:
print range(5)
print range(5,9)
print range(9,15,2)
print range(15,9,-2)
print zip(range(5),range(5,9))
print 3 in [8,5,6,4,3,2]
print len([4,6])
print [1,2,3]+[2,4,6]

# Extract parts of a list:
a = ['a','b','c','d','e','f']
print a[0], a[3], a[-2]
print a[1:3], a[3:-1], a[4:]
```

```
[0, 1, 2, 3, 4]
[5, 6, 7, 8]
[9, 11, 13]
[15, 13, 11]
[(0, 5), (1, 6), (2, 7), (3, 8)]
True
2
[1, 2, 3, 2, 4, 6]

a d e
['b', 'c'] ['d', 'e'] ['e', 'f']
```


Basics: Lists are mutable

```
—  
# Lists are mutable:  
a = [1,2,3,4,5]; print a
```

```
a.append(10)  
print a
```

```
print a.append(9)  
print a
```

```
a[1:3]=[7,7,7,7]  
print a,
```

```
a[4:8]=[]  
print a
```

```
# Ex of more mutating methods:  
#  
#     extend, insert, pop,  
#     sort, reverse  
#
```

```
[1, 2, 3, 4, 5]
```

```
[1, 2, 3, 4, 5, 10]
```

```
None
```

```
[1, 2, 3, 4, 5, 10, 9]
```

```
[1, 7, 7, 7, 7, 4, 5, 10, 9]
```

```
[1, 7, 7, 7, 9]
```

Basics: Handle dictionaries

```
—  
# Handle dictionaries:  
a = {1: 'A', 2: 'B'}  
print a.keys()  
print a.values()  
print a.items()  
print 1 in a  
print 'A' in a  
print len({1:1,4:7})  
  
# Dictionaries are mutable:  
a.update({1:123, 3: 'C'})  
print a  
  
a[8] = 345  
print a  
  
# Ex of more mutating methods:  
#  
#     clear, pop  
#
```

```
[1, 2]  
['A', 'B']  
[(1, 'A'), (2, 'B')]  
True  
False  
2  
  
{1: 123, 2: 'B', 3: 'C'}  
  
{8: 345, 1: 123, 2: 'B', 3: 'C'}
```

Basics: Case matters

```
—  
# Case matters:
```

```
hi = 1
```

```
Bye = Hi
```

```
Traceback (most recent call last):  
  File "Basics_01d.py", line 6, in <m  
    Bye = Hi  
NameError: name 'Hi' is not defined
```



Run

Clear



Basics: Indentation is syntax

```
—  
# Indentation is syntax:  
for i in [3,4]:  
    print "Within the for loop:"  
    print i  
  
—  
i=1  
while i<3:  
    print "Within the while loop:"  
    print i  
    i+=1  
  
—  
if 2<3:  
    print "2 is less than 3."  
elif 2==3:  
    print "2 equals 3."  
else:  
    print "2 is larger than 3."
```

```
Within the for loop:  
3  
Within the for loop:  
4  
  
Within the while loop:  
1  
Within the while loop:  
2  
  
2 is less than 3.
```

Basics: Variables are not declared

```
—  
# Variables are not declared:  
hi = 1  
print hi  
  
# Variables can change type:  
hi = '- Oops!'  
print hi
```

1

- Oops!



Run

Clear



Basics: Assignment does not copy

```
—  
# Assignment does not copy:
```

```
a = [1,2,3]; print a
```

```
b = a; print b
```

```
print a==b, a is b
```

```
a.append(4)
```

```
print a,b
```

```
print a==b, a is b
```

```
# Creating a copy:
```

```
a = [1,2,3]; print a
```

```
b = a[:]; print b
```

```
print a==b, a is b
```

```
a.append(4)
```

```
print a,b
```

```
print a==b, a is b
```

```
[1, 2, 3]
```

```
[1, 2, 3]
```

```
True True
```

```
[1, 2, 3, 4] [1, 2, 3, 4]
```

```
True True
```

```
[1, 2, 3]
```

```
[1, 2, 3]
```

```
True False
```

```
[1, 2, 3, 4] [1, 2, 3]
```

```
False False
```

Basics: Arithmetic

```
—  
# Arithmetic:  
print 3 + 4*2 - 1  
print complex(12,8)/3.2  
print 7 % 4          # Modulo  
  
—  
# Obs!  
print 7^5            # Bit-wise XOR  
print 7**5           # Exponentiation  
  
—  
# More logical operators  
print 6 & 3          # Bit-wise AND  
print 6 | 3          # Bit-wise OR  
print ~6             # Bit-wise NOT
```

```
10  
(3.75+2.5j)  
3  
  
2  
16807  
  
2  
7  
-7
```



Run

Clear



Basics: Even more logic

```
—  
# Even more logic:  
F,T = False,True  
print F,T, type(F), type(T)  
print T+T, type(T+T)  
print int(F),int(T)  
  
print not F, not T  
  
print F and F, F and T  
print T and F, T and T  
  
print F or F, F or T  
print T or F, T or T  
  
from operator import xor  
print xor(F,F), xor(F,T)  
print xor(T,F), xor(T,T)
```

```
False True <type 'bool'> <type 'bool'>  
2 <type 'int'>  
0 1  
  
True False  
  
False False  
False True  
  
False True  
True True  
  
False True  
True False
```


Basics: Logic using integers

```
# Logic using integers:  
F,T = 0,1  
print F,T, type(F), type(T)  
print T+T  
print int(F),int(T)  
  
print not F, not T  
  
print F and F, F and T  
print T and F, T and T  
  
print F or F, F or T  
print T or F, T or T  
  
from operator import xor  
print xor(F,F), xor(F,T)  
print xor(T,F), xor(T,T)
```

```
0 1 <type 'int'> <type 'int'>  
2  
0 1  
  
True False  
  
0 0  
0 1  
  
0 1  
1 1  
  
0 1  
1 0
```

Basics: Everything has a logical meaning

```
—  
# Everything has a logical meaning:  
print bool(0), bool(1)  
print bool(0.0), bool(0.1)  
print bool(0j), bool(1j)  
print bool(()), bool((0,2))  
print bool([]), bool([0])  
print bool({}), bool({1:'a'})  
  
# _Everything_ has a logical meaning:  
print [] and 234  
print [0] and 234  
print [] or 234  
print [0] or 234  
  
print not []  
  
if []:  
    print 'The list is not empty.'  
else:  
    print 'The list is empty.'
```

```
False True  
False True  
False True  
False True  
False True  
False True
```

```
[]  
234  
234  
[0]
```

```
True
```

```
The list is empty.
```



Run

Clear



Basics: Mixed types

```
—  
# Mixed types:  
print 2*2.0  
print 2*'abc'  
print 2*[1,2,3]  
  
# Casting types:  
print int(2.5)  
print float(4)  
print list('abc')  
print ' and '.join(['A','B','C'])
```

```
4.0  
abcab  
[1, 2, 3, 1, 2, 3]  
  
2  
4.0  
['a', 'b', 'c']  
A and B and C
```



Run

Clear



Basics: List comprehensions

```
—  
  
# List comprehensions:  
a = range(6)  
b = [[x] for x in a]  
print b  
  
print [x.append(x[0]**2) for x in b]  
  
print b  
  
print [x for x in a if x>2]  
  
print [  
    (x,y)  
    for x in a if x>1  
    for y in a if y>x and y % 2  
    ]
```

```
[[0], [1], [2], [3], [4], [5]]  
  
[None, None, None, None, None, None]  
  
[[0, 0], [1, 1], [2, 4], [3, 9], [4,  
[3, 4, 5]  
  
[(2, 3), (2, 5), (3, 5), (4, 5)]
```

Basics: The pitfall division, part 1

```
—  
# The pitfall division, part 1:  
print 4/3  
print 4/3.0  
print 4.0/3  
print 4.0/3.0  
  
print 4//3  
print 4//3.0  
print 4.0//3  
print 4.0//3.0
```

```
1  
1.333333333333  
1.333333333333  
1.333333333333  
  
1  
1.0  
1.0  
1.0
```



Run

Clear



Basics: The pitfall division, part 2

```
# The pitfall division, part 2:  
  
from __future__ import division  
  
_  
print 4/3  
print 4/3.0  
print 4.0/3  
print 4.0/3.0  
  
print 4//3  
print 4//3.0  
print 4.0//3  
print 4.0//3.0
```

```
1.333333333333  
1.333333333333  
1.333333333333  
1.333333333333  
  
1  
1.0  
1.0  
1.0
```



Run

Clear



Basics: Functions

```
—  
# Functions:  
def gcd(a,b):  
    '''Return the greatest common  
    divisor of a and b.'''  
    while b != 0:  
        a, b = b, a % b  
    return a  
  
print gcd(120,640)
```

```
# Documentation:  
help(gcd)
```

40

Help on function gcd:

```
gcd(a, b)  
    Return the greatest common  
    divisor of a and b.
```



Run

Clear



Basics: Mathematical functions

```
—  
# Mathematical functions:  
print abs(complex(3,4))  
# arg is missing!!!  
print cmp(2,4), cmp(3,3), cmp(4,2)  
print divmod(7,4)  
print pow(3,5)      # == 3**5  
print pow(3,5,7)    # == 3**5 % 7  
  
—  
print oct(378), hex(378)  
  
print max(7,3,5), min([7,3,5])  
print max('jysghgds')  
print sum([2,3,4]), sum([2,3,4],10)  
  
print int(3.456789)  
print round(3.4567), round(3.4567,2)
```

```
5.0  
  
-1 0 1  
(1, 3)  
243  
5  
  
0572 0x17a  
  
7 3  
y  
9 19  
  
3  
3.0 3.46
```


Basics: Mathematical functions

```
—  
# Mathematical functions:  
print abs(complex(3,4))  
# arg is missing!!!  
print cmp(2,4), cmp(3,3), cmp(4,2)  
print divmod(7,4)  
print pow(3,5)      # == 3**5  
print pow(3,5,7)    # == 3**5 % 7  
  
—  
print oct(378), hex(378)  
  
print max(7,3,5), min([7,3,5])  
print max('jysghgds')  
print sum([2,3,4]), sum([2,3,4],10)  
  
print int(3.456789)  
print round(3.4567), round(3.4567,2)
```

```
5.0  
  
-1 0 1  
(1, 3)  
243  
5  
  
0572 0x17a  
  
7 3  
y  
9 19  
  
3  
3.0 3.46
```

Classes: Object oriented? (1)

```
—  
# Object oriented? (1)  
class empty:  
    pass  
  
a=empty()  
  
print a.foo
```

```
Traceback (most recent call last):  
  File "Classes_01.py", line 8, in <m  
    print a.foo  
AttributeError: empty instance has no
```



Run

Clear



Classes: Object oriented? (2)

```
-  
# Object oriented? (2)  
class empty:  
    pass  
  
a=empty()  
a.foo = '- Ha!'  
print a.foo
```

- Ha!



Run

Clear



Classes: Classes

```
—  
# Classes  
class Paradise:  
    print "Introducing Paradise!"  
    def him(self):  
        return 'Adam'  
  
    def her(self):  
        return 'Eve'  
  
    def it(self):  
        return 'The serpent'  
  
a=Paradise()  
print a.him()  
print a.her()  
print a.it()
```

```
Introducing Paradise!  
Adam  
Eve  
The serpent
```



Run

Clear



Classes: Inheritance

```
—  
# Inheritance  
class Paradise:  
    print "Introducing Paradise!"  
    def him(self):  
        return 'Adam'  
    def her(self):  
        return 'Eve'  
    def it(self):  
        return 'The serpent'  
  
class Paradise2(Paradise):  
    print "Introducing another Paradise"  
    def it(self):  
        return 'The Porche'  
  
b=Paradise2()  
print b.him()  
print b.her()  
print b.it()
```

```
Introducing Paradise!  
Introducing another Paradise!  
Adam  
Eve  
The Porche
```



Run

Clear



Classes: Magical methods

```
—  
# Magical methods  
class line:  
    def __init__(self,a,m):  
        self.a = a  
        self.m = m  
    def __call__(self,x):  
        return self.a*x + self.m  
    def __add__(self,other):  
        aa = self.a + other.a  
        mm = self.m + other.m  
        return line(aa,mm)  
    def __repr__(self):  
        am = (self.a,self.m)  
        return 'line(%s,%s)' % am  
y1 = line(2,3)  
print y1.a, y1.m  
print y1(1), y1(2), y1(3)  
y2 = line(5,7)  
y = y1+y2  
print y
```

```
2 3  
5 7 9
```

```
line(7,10)
```



Run

Clear



Classes: Comparing elements 1

```
—  
# Comparing elements 1  
class poor:  
    def __init__(self,a):  
        self.data = a  
  
    def __cmp__(self,other):  
        return self.data - other.data  
  
a,b = poor(4), poor(6)  
print a.__cmp__(b)  
print cmp(a,b)  
print a<b, a<=b  
print a>b, a>=b  
print a==b, a!=b
```

```
-2  
-1  
True True  
False False  
False True
```



Run

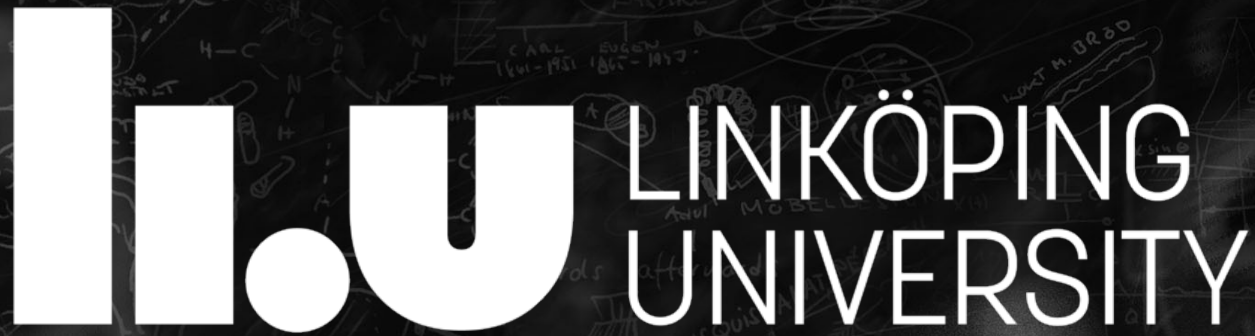
Clear



Classes: Comparing elements 2

```
—  
# Comparing elements 2  
class rich:  
    def __lt__(self,other):  
        return True  
    def __le__(self,other):  
        return False  
    def __gt__(self,other):  
        return True  
    def __ge__(self,other):  
        return False  
    def __eq__(self,other):  
        return True  
    def __ne__(self,other):  
        return True  
  
a,b = rich(),rich()  
print a<b, a<=b  
print a>b, a>=b  
print a==b, a!=b
```

```
True False  
True False  
True True
```

www.liu.se